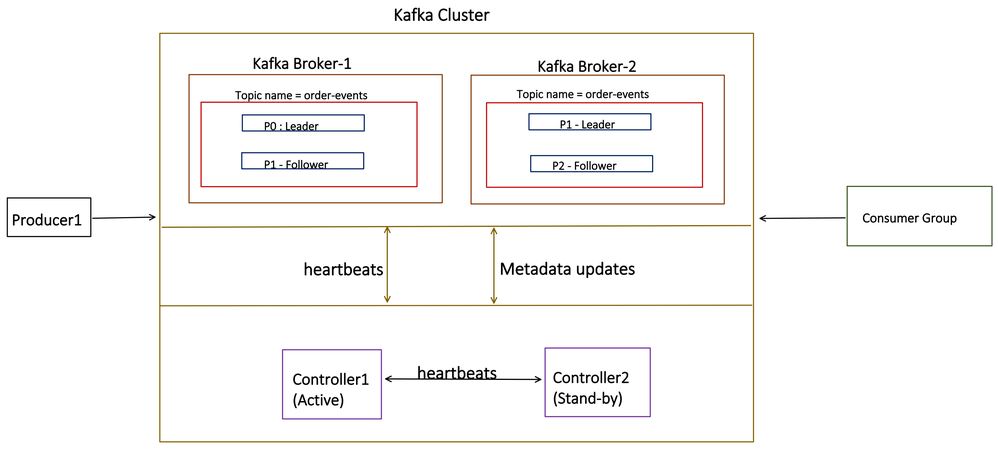
7. Cluster Setup

Thursday, 26 February 2026 7:07 PM

Print

Today, will setup below Kafka Cluster.



Step1: Download Binary of latest Kafka Version

Note:
Instead of downloading Binary, we can also go with Kafka Docker Image. But since I haven't covered Docker yet, for now will show how to do with Binary download.

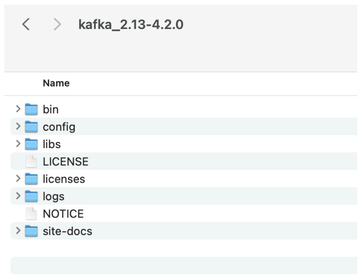
Which is also good way to understand things and once understood properly, Running via Docker image is no different and would be very easy.

Lets download latest Kafka Binary version from : https://kafka.apache.org/community/downloads/

I choose 4.2.0, Which is latest and released on 17th Feb 2026

Binary download: kafka_2.13-4.2.0.tgz (asc, sha512)

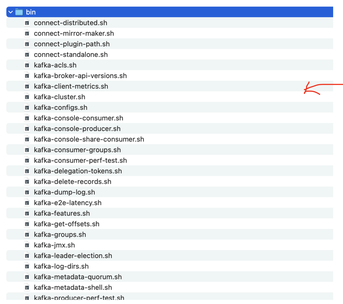
Once downloaded, you will see below sub-folders within Kafka folder.



Print

Step2: Understand each sub-folder

Bin:



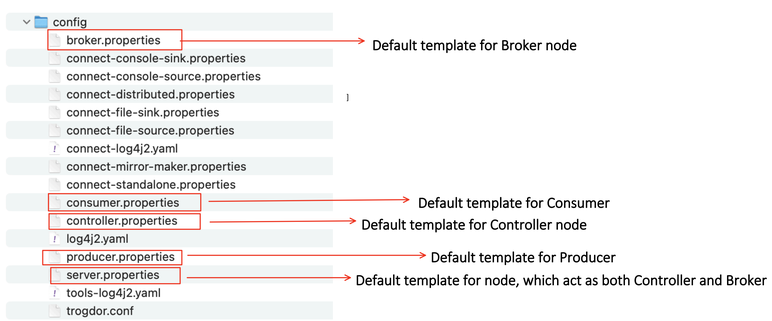
This folder contains executable shell scripts, which is used to perform tasks like:

- Start controller server
- Start broker server
- Create topic
- Delete topic
- Publish message
- Consume message

etc..

These scripts are wrappers around Kafka internal Java APIs. So that we can run and manage kafka components from CLI itself.

Config: This folder is important for us, as it contains all the configuration for our Controller and Broker nodes.



Default template for Broker node

Default template for Consumer

Default template for Controller node

Default template for Producer

Default template for node, which act as both Controller and Broker

Other folders are like:

libs: contains the kafka jar files and their dependencies. Scripts within bin/ uses these JARs.

logs: Contains Kafka runtime/application logs (INFO, WARN, ERROR messages) generated when brokers or controllers start and run.

Step3: Write the configuration for Controllers

These configuration file need to put in /config folder.

controller1.properties

```
process.roles=controller
node.id=1
listeners=CONTROLLER://:9093
controller.listener.names=CONTROLLER
controller.quorum.voters=1@localhost:9093,2@localhost:9193
# PLAINTEXT (no encryption). In production, we should use SSL.
listener.security.protocol.map=CONTROLLER:PLAINTEXT
log.dirs=/tmp/controller1-logs
```

controller2.properties

```
process.roles=controller
node.id=2
listeners=CONTROLLER://:9193
controller.listener.names=CONTROLLER
controller.quorum.voters=1@localhost:9093,2@localhost:9193
# PLAINTEXT (no encryption). In production, we should use SSL.
listener.security.protocol.map=CONTROLLER:PLAINTEXT
log.dirs=/tmp/controller2-logs
```

```
# Default partitions when creating a topic without specifying
num.partitions=3
default.replication.factor=2
```

```
# Default partitions when creating a topic without specifying
num.partitions=3
default.replication.factor=2
```

process.roles -> tells kafka what role this node should perform:

Roles:
- controller : only perform controller tasks
- broker : only perform broker tasks
- controller,broker : performs tasks of both controller and broker.

node.id -> unique id for the node within a cluster.

listeners -> list of Port which are running on this node, also for each port we can give logical name.
internally it does something like:
new ServerSocket(9093), a TCP socket waiting for connections.

listeners=CONTROLLER://:9093,BROKER://:9092

This tells Kafka, open 2 sockets:
- One named CONTROLLER
- One named BROKER

listeners=CONTROLLER://:9093

This is just a logical name (label) for this socket.

If IP is omitted, kafka will listen on every IP address on the machine.
if you want any specific IP address: CONTROLLER://192.168.1.10:9093

controller.listener.names ->

listeners=CONTROLLER://:9093, INTERNAL://:9092, EXTERNAL://:9094
controller.listener.names=CONTROLLER

This tells, among all the listeners defined, use the one with named "CONTROLLER" for controller-controller communication.

controller.quorum.voters -> each controller, maintains a static list of nodes which act as controller

controller.quorum.voters=1@localhost:9093,2@localhost:9193

listener.security.protocol.map -> For which listener, what security protocol need to be used.

PLAINTEXT - no encryption, no authentication, used for local
SSL - TLS encryption, used in production

listener.security.protocol.map=CONTROLLER:PLAINTEXT

For controller listener, use PLAINTEXT security protocol.

log.dirs -> directory where node stores its data.

num.partitions -> while during topic creation, if producer or CLI do not tell how many partitions, then controller use this default value.
default.replication.factor -> default value for RF, how many replication for each partition.

offsets.topic.replication.factor -> default value for _consumer_offset topic

Step4: Write the configuration for Brokers

These configuration file need to put in /config folder.

broker1.properties

```
process.roles=broker

node.id=3

# WHERE to fetch metadata from (which controllers to connect to)
controller.quorum.voters=1@localhost:9093,2@localhost:9193

# This is where our Spring Boot apps(producer/consumer) will connect
listeners=BROKER://:9092

# passed this address to controller to store in their cluster metadata
advertised.listeners=BROKER://localhost:9092

# internally when follower broker try to fetch from leader, what listener they need to use.
inter.broker.listener.name=BROKER


# when broker try to communicate with controller what listener they need to use.
controller.listener.names=CONTROLLER

listener.security.protocol.map=CONTROLLER:PLAINTEXT,BROKER:PLAINTEXT

log.dirs=/tmp/broker1-logs

# Keep messages for 7 days (168 hours)
log.retention.hours=168

# Max size of a single log segment file (1GB)
log.segment.bytes=1073741824

# __consumer_offsets topic RF
offsets.topic.replication.factor=2
```

Each broker also maintains the static list of controller nodes

Opens up the port and named it BROKER.

This address is passed to controller. And controller stored this address in its metadata.

when producer want to push to Topic-A and Partition0,
it will ask any Broker -> Broker goes to Controller, Controller returns the metadata,

which has all the details like which broker is the leader of which topic partition.
and this address, so that producer can directly call that specific Broker.

This tells, among all the listeners defined, use the one with named "BROKER" for broker-broker communication.

As we know follower broker talk with leader broker to fetch the data.

Directory in which this node data will persist.

2 use:
- Broker use this Controller listener name to map the Security protocol.

- Fetch the controller listener from the metadata

lets say in cluster metadata this info is present:

```
{
    "controllerId": 1,
    "endPoints": [
        {
            "name": "CONTROLLER",
            "host": "localhost",
            "port": 9093,
            "securityProtocol": 0
        },
        {
            "name": "INTERNAL",
            "host": "localhost",
            "port": 9097,
            "securityProtocol": 1
        }

    ]
}
```

say broker has decided, it need to communicate with Controller with ID:1 (as its ACTIVE), so it will pick the listener with name "CONTROLLER"

And that's one reason why all different controllers have same listener name.

Kafka will delete the whole segments if its first event is older than 168 hours (7 days).

Each segment size is fixed = 1GB
After that new segment is created for the partition

Similarly, broker2.properties

```
process.roles=broker
```

```
node.id=4

# WHERE to fetch metadata from (which controllers to connect to)
controller.quorum.voters=1@localhost:9093,2@localhost:9193

# This is where our Spring Boot apps(producer/consumer) will connect
listeners=BROKER://:9192

# passed this address to controller to store in their cluster metadata
advertised.listeners=BROKER://localhost:9192

# internally when follower broker try to fetch from leader, what listener they need to use.
inter.broker.listener.name=BROKER


# when broker try to communicate with controller what listener they need to use.
controller.listener.names=CONTROLLER

listener.security.protocol.map=CONTROLLER:PLAINTEXT,BROKER:PLAINTEXT

log.dirs=/tmp/broker2-logs

# Keep messages for 7 days (168 hours)
log.retention.hours=168

# Max size of a single log segment file (1GB)
log.segment.bytes=1073741824

# __consumer_offsets topic RF
offsets.topic.replication.factor=2
```

Step5: map all these nodes ids (2 broker and 2 controller) with 1 kafka Cluster

Generate cluster ID using script present in "/bin" folder:

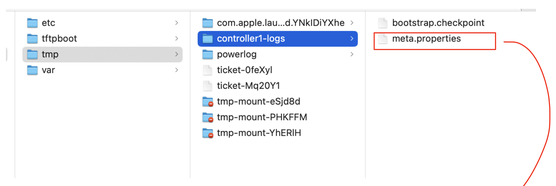
bin/kafka-storage.sh random-uuid

Sample output: zJVVVZAkS9aIBfjY0BlYJA

Map each node with this cluster id:

1. **Controller1:**
   bin/kafka-storage.sh format -t *zJVVVZAkS9aIBfjY0BlYJA* -c config/controller1.properties
2. **Controller2:**
   bin/kafka-storage.sh format -t *zJVVVZAkS9aIBfjY0BlYJA* -c config/controller2.properties
3. **Broker1:**
   bin/kafka-storage.sh format -t *zJVVVZAkS9aIBfjY0BlYJA* -c config/broker1.properties
4. **Broker2:**
   bin/kafka-storage.sh format -t *zJVVVZAkS9aIBfjY0BlYJA* -c config/broker2.properties

This will create "meta.properties" file within each log.dirs path.




```
#
#Thu Feb 26 22:04:56 IST 2026
cluster.id=zJVVVZAkS9aIBfjY0BlYJA
directory.id=LYSuEbfGps8gnV7YsC9JHA
node.id=1
version=1
```

Print

Step6: Start all the node servers

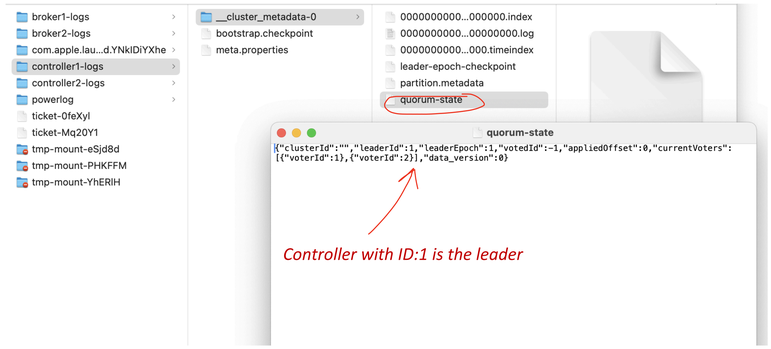
Order matters like : Controller first and then Broker. As broker try to call Controller to load the cluster metadata information.

Controller1:

bin/kafka-server-start.sh config/controller1.properties

Controller2:

bin/kafka-server-start.sh config/controller2.properties



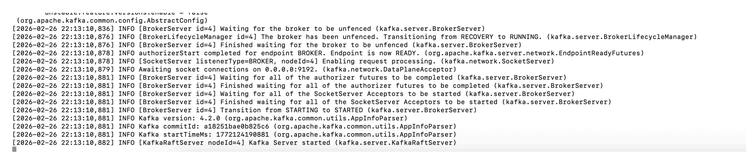
Broker1:

bin/kafka-server-start.sh config/broker1.properties



Broker2:

bin/kafka-server-start.sh config/broker2.properties



Check the cluster status:

bin/kafka-metadata-quorum.sh --bootstrap-controller localhost:9093 describe --status



controllers *brokers*

Print

Step7: All setup completed, now lets test the Producer and consumer using script.

Lets create sample topic to test:

In this we are not passing, how many partitions and Replication factor, so Active controller use its default.

bin/kafka-topics.sh --bootstrap-server localhost:9092 --create --topic order-events-topic

--bootstrap-server localhost:9092, is only required for 1st time, because request need to go to some broker and that will ultimately pass to Leader Controller.

Once done, we can check the status of topic:

bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic order-events-topic

```
shrayanshjain@Shrayanshs-MacBook-Pro kafka_2.13-4.2.0 % bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --describe --topic order-events-topic
Topic: order-events-topic       TopicId: FGBARB40QkC4MbwWuaNk2A PartitionCount: 3       ReplicationFactor: 2    Configs: min.insync.replicas=1,segment.bytes=1073741824
        Topic: order-events-topic       Partition: 0    Leader: 3       Replicas: 3,4   Isr: 3,4        Elr:    LastKnownElr:
        Topic: order-events-topic       Partition: 1    Leader: 4       Replicas: 4,3   Isr: 4,3        Elr:    LastKnownElr:
        Topic: order-events-topic       Partition: 2    Leader: 3       Replicas: 3,4   Isr: 3,4        Elr:    LastKnownElr:
```

Step8: Producer test

bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic order-events-topic

```
shrayanshjain@Shrayanshs-MacBook-Pro kafka_2.13-4.2.0 % bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic order-events-topic

>hello
>this is shrayansh
>how are you
>
```

Step9: Consumer test

bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic order-events-topic --from-beginning

```
shrayanshjain@Shrayanshs-MacBook-Pro kafka_2.13-4.2.0 % bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic order-events-topic --from-beginning

hello
this is shrayansh
how are you
```

Print